Project 1: Build and Evaluate a Small Code Language Model

CS 5090 · Software Engineering with Generative AI

Individual Project · 25% of Course Grade · Due Friday, October 30

1. Overview

In this project, you will train a small code language model from scratch, use it on a software engineering task, study how it fails, and make one evidence-based improvement. The goal is not to build a strong production model. The model is intentionally small so that you can understand the full workflow and inspect its behavior.

Phase	Recommended pace	Main task
1. Build & understand	by Sep 25	Train a small code LM and run one ablation
2. Use	by Oct 9	Use your model on 20 real SE inputs
3. Evaluate & break	by Oct 16	Design probes and document 10+ failure instances
4. Improve & re-evaluate	by Oct 23	Make one evidence-based intervention and re-run the same probes

Final submission

Submit only two main items on Friday, October 30:

- Final report (recommended length: 6–8 pages, excluding appendices).
- Code repository with everything needed to reproduce your baseline, ablation, evaluation, and Phase 4 intervention.

Put supporting material in the report appendices or repository rather than creating separate reports. This includes the findings table, AI-use log, configuration details, and the handwritten reflection. A scan/photo of the handwritten reflection may be included as a report appendix.

> *The intermediate dates are recommended milestones, not separate deadlines.*

2. Phase 1 — Build & Understand

Train a small GPT-style model from scratch. You may use either nanoGPT (recommended) or llama2.c. Find your own Python corpus, for example a few permissively licensed repositories whose code you know well enough to judge; different corpora across the class make the results more diverse. Keep collection light (a few hours, not a project of its own), and document in your data sheet how the data was filtered, deduplicated, tokenized, and split.

Example model configuration

The following is one reasonable starting point. You may adjust these settings based on your codebase, compute environment, or experimental goals. Document major changes in your report.

Setting	Example value
Layers / heads / embedding size	6 / 6 / 384
Context length	512 tokens
Tokenizer	BPE, vocabulary 8,192; train only on the training split
Training tokens	approximately 20 × parameter count
Precision	bf16 when supported

Ablation study

After the baseline model trains, run one ablation. The options below are recommended; you may propose a comparable alternative with instructor approval. Write down your prediction before the run, then compare the ablated model with the baseline using final validation loss and at least one side-by-side generation example.

Recommended option	Question it helps you study
Remove position embeddings	Why does token order need to be represented?
Use 1 attention head instead of 6	What is gained by multiple heads?
Use 3 layers instead of 6	What is gained by depth?
Remove LayerNorm	How does normalization affect training stability?
Context length 256 instead of 512	What does a longer context help with?
Character-level tokenizer instead of BPE	How does tokenization change training and generation?

What you should understand

- How source code becomes tokens and why tokenizer choice matters.
- How embeddings and self-attention provide contextual token representations.
- What next-token prediction and cross-entropy loss optimize.
- The difference between pre-training and inference.
- How to interpret training and validation loss curves.
- Why a small model trained on a small corpus can produce fluent but incorrect code.

What goes in the final report

- Architecture/configuration, training setup, and key data-processing choices.
- Training and validation results (include a loss curve).
- Ablation prediction, exact change, result, and what you learned.
- A short discussion of limitations and data/licensing considerations.

If your training run fails after reasonable effort, contact the instructor about the spare checkpoint. You can still complete Phases 2–4, but missing Phase 1 work will not receive full credit.

3. Phase 2 — Use the Model on a Real SE Task

Use your own model on one software engineering task, such as code completion, docstring generation, test skeleton generation, or another task approved by the instructor. Use 20 real inputs from code you understand well enough to judge.

For each input, record only a brief decision: accept as-is, accept after editing, or reject. You do not need a long write-up for all 20 inputs. Select three examples for deeper discussion: the best result, the worst result, and one near-miss that initially looked correct but was not.

Outcome	Count
Accept as-is	
Accept after editing	
Reject	

What goes in the final report

- The SE task, source of the 20 inputs, and what “accept” means for your task.
- Accept/edit/reject counts.
- The best, worst, and near-miss examples with short analysis.
- A brief assessment of when the model was useful and when checking it cost more time than doing the work yourself.

4. Phase 3 — Evaluate & Break

Build a probe set of approximately 20–30 inputs designed to expose weaknesses in your model. Include ordinary inputs and targeted cases such as misleading comments or names, long-context inputs, security-related prompts, prompt variations, or prompts similar to the training data.

Document at least 10 failure instances. Try to cover multiple failure categories (four or more where the model behavior allows), rather than collecting many minor variations of the same mistake. Use the category names below exactly so results can be compared across the class.

Category	Definition
syntax	Output does not parse
invented-api	Calls an API or function that does not exist
wrong-logic	Parses/runs but produces the wrong behavior
insecure	Introduces a security weakness
memorised	Reproduces training data nearly verbatim
prompt-sensitive	A small rewording changes the answer substantially
context-ignored	Ignores information clearly present in the input
other	Use only when none of the above applies; explain clearly

For each failure, record the input, relevant output, category, severity, a specific one-line explanation of why it is wrong, how you found it, and whether a normal unit-test suite would likely catch it. Keep the full findings table as a CSV in your repository; summarize the main patterns in the report.

What goes in the final report

- How you designed the probe set and what behaviors you targeted.
- A summary of the failure distribution and the most important examples.
- Your interpretation of likely root causes.
- What fraction/types of failures ordinary tests would and would not catch.

5. Phase 4 — Improve & Re-evaluate

Use your Phase 3 evidence to select one important failure pattern. Form a hypothesis about its cause, make one intervention, and re-run the same probes. The intervention does not have to modify or retrain the model itself. In AI-for-SE, improving the surrounding software system may be the better engineering choice.

Intervention area	Examples
Data	Remove unparseable files, improve deduplication, rebalance data
Model / training	Change context, training schedule, or another justified configuration
Fine-tuning	A small supervised fine-tuning pass on relevant examples
Decoding / prompting	Adjust decoding, prompt structure, or candidate selection
Software-system layer	Syntax/static checks, unit-test filtering, retrieval, retry/self-correction, post-generation verification

Experimental rules

1. State your hypothesis before running the experiment.
2. Change one main thing so the result is interpretable.
3. Re-run the same Phase 3 probes under comparable conditions.
4. Report improvements and regressions, not only wins.
5. Report compute cost and validation loss when they are relevant.
6. A negative result is acceptable if the experiment is well designed and the evidence is reported clearly.

What goes in the final report

- Target failure pattern and hypothesis.
- The intervention and why it follows from your Phase 3 evidence.
- Before/after results on the same probes.
- Regressions, costs, and threats to validity.
- Whether the intervention worked for the reason you expected, and what you would try next.

6. AI-Use Log and Reflection

You may use an AI coding assistant while completing the project. Keep a lightweight log of representative, meaningful interactions rather than every completion. Include examples of where AI saved time, where it was confidently wrong, what you had to fix, and how you verified its suggestions.

Include the AI-use log as a report appendix or repository file. Also complete a one-page handwritten reflection without AI assistance and include a scan/photo as a report appendix. The reflection is graded for honest completion, not for a particular answer.

Suggested reflection questions

7. What can you explain now that you could not explain before this project?
8. Did building and testing your own model change how you judge tools such as Copilot or ChatGPT? How?
9. Which phase taught you the most, and why?
10. What about language models still confuses you?

7. Suggested Final Report Structure

Recommended length: 6–8 pages, excluding appendices. You may organize the report differently if the same information is easy to find.

Section	Suggested content
1. Build & Understand	Model/data setup, training results, ablation, limitations
2. Use	Task, 20-input results, best/worst/near-miss
3. Evaluate & Break	Probe design, failure summary, root-cause analysis, testability
4. Improve & Re-evaluate	Hypothesis, intervention, before/after results, regressions/cost
5. Discussion	What you learned, threats to validity, what you would try next
Appendices	Full findings table if desired, AI-use log, handwritten reflection, extra plots/configs

8. Grading

Model quality itself is not graded. You are graded on understanding, experimental design, evidence, analysis, and reproducibility. A well-supported negative result can receive full credit.

Component	Weight	Full credit requires
Build & understand	25%	Reproducible baseline + ablation; prediction recorded before ablation; clear understanding of model/data/training choices
Use	15%	20 real inputs; honest accept/edit/reject judgments; informative representative examples
Evaluate & break	25%	10+ failure instances; systematic probe design; consistent categories; specific analysis and

Component	Weight	Full credit requires
		testability assessment
Improve & re-evaluate	25%	Clear hypothesis; one justified intervention; same probes re-run; before/after evidence; regressions/costs/threats reported
Report, AI-use reflection & engineering quality	10%	Concise final report; meaningful AI-use log; handwritten reflection; repository is reproducible and results are traceable

9. Compute

A GPU is strongly recommended for Phase 1 training. A successful run may take roughly 2–6 GPU-hours depending on the environment. As a planning guideline, you are not expected to spend more than about 15 GPU-hours across the project. Kaggle, Google Colab, or a local GPU are acceptable. Save checkpoints regularly and log training loss so runs can be reproduced and analyzed later.

If an ACCESS classroom allocation becomes available, instructions will be posted separately. The project does not depend on ACCESS availability.

10. Repository Checklist

- README with setup and reproduction instructions.
- Environment/dependency specification and codebase commit/version.
- Data preparation or data-rebuild instructions.
- Baseline and ablation configurations.
- Phase 2 input/result records.
- Phase 3 probe set and findings CSV.
- Phase 4 intervention code/configuration and before/after results.
- AI-use log if not included in the report appendix.